C11 Input System

CRE132 worksheet

What you'll make

Two identical heroes side by side. The left one runs C05 and C06's scripts, which read keys with the old **Input Manager**. You'll make the right one do exactly the same with Unity's newer **Input System**, so it also works with a gamepad.

You'll build an **Input Actions** asset with a **Move** and a **Jump** action, give them keyboard and gamepad **bindings**, and rewrite the two scripts so they read actions instead of keys. Only the input lines change; the physics code stays the same. **From now on, every sample uses the Input System.**

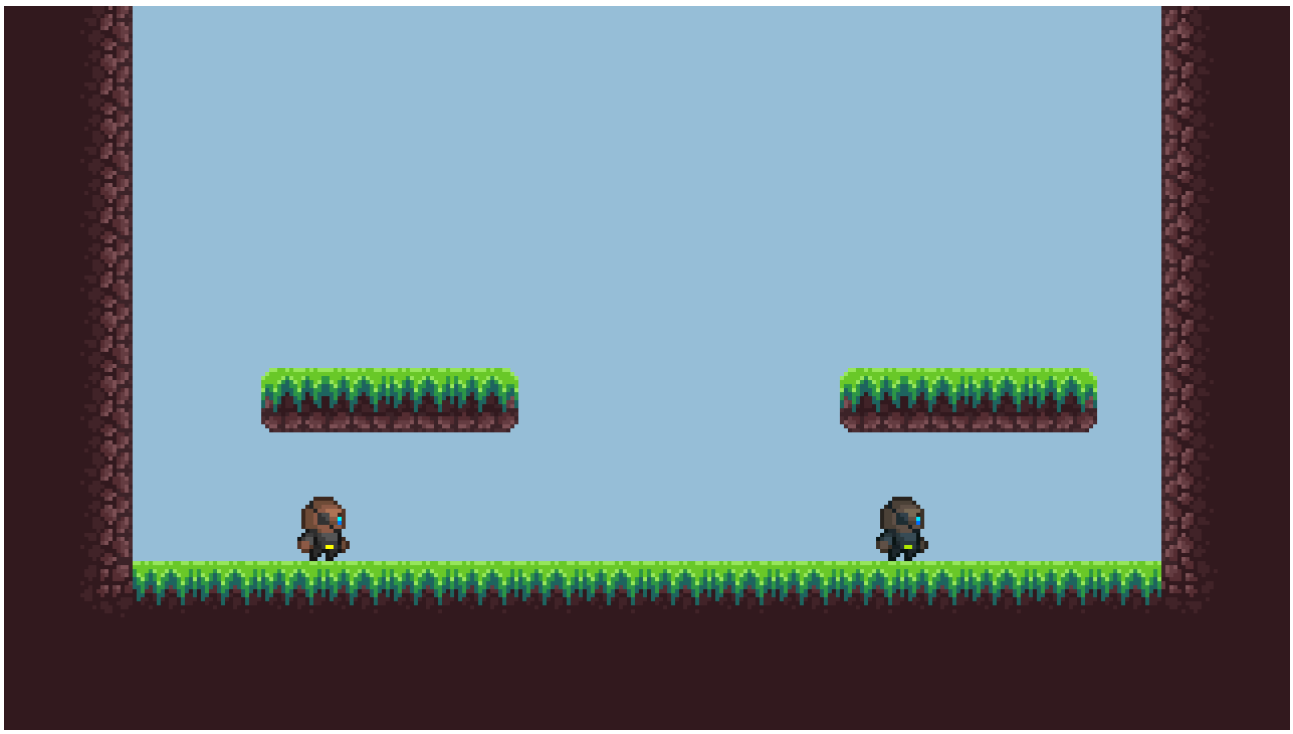


Figure 1: The finished scene: two heroes on the ground between two cliffs, with a floating platform above each.

Before you start

1. Unzip `CRE132_C11_InputSystem_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C11_InputSystem_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C11_InputSystem`.
3. Press **Play**. **Player (Input Manager)** on the left walks with A/D or the arrows and jumps with Space. **Player (Input System)** on the right, tinted blue, doesn't move yet. Press Play again to stop.

Why switch? With the Input System, one action can have many bindings (keyboard, gamepad, touch), players can rebind keys, and it's Unity's current system. This project's **Active Input Handling** (in **Edit > Project Settings > Player**) is set to **Both**, so the old and new systems work side by side.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The Input Actions asset

Make a folder `Assets/_game/Input`. Right-click in it and choose **Create > Input Actions**. Name it `CRE132Controls` and double-click it to open the Input Actions editor.

It has three columns: **Action Maps**, **Actions** and **Action Properties**.

1. **Action map:** click **+** next to Action Maps and name the map `Player`. A map is a set of actions used together (later samples could add a `UI` map).
2. **Move:** click **+** next to Actions and name it `Move`. In Action Properties, set **Action Type** to **Value** and **Control Type** to **Vector2**: a direction with an x and a y, like C04's.
3. **Jump:** add another action called `Jump`. Leave its Action Type as **Button**.

- **Actions** describe *what* the player wants to do (Move, Jump).
- **Bindings** say *which keys or buttons* do it. Your code will only ever read actions.

2. Bindings

Move: click the **+** on the Move action and choose **Add Up\Down\Left\Right Composite**. Name it `WASD`. Click each of its four parts (**Up**, **Down**, **Left**, **Right**), click the **Path** drop-down and press **Listen**, then press the key: W, S, A and D. Add a second composite called `Arrows` with the four arrow keys.

Then add two more bindings to Move with **+ > Add Binding**: set one's Path to **Left Stick [Gamepad]** and the other's to **D-Pad [Gamepad]**. Both give a Vector2, so they need no composite.

Jump: select the binding under Jump and set its Path to **Space [Keyboard]**. Add a second binding with the Path **Button South [Gamepad]** (the bottom face button: A on Xbox, Cross on PlayStation).

Click **Save Asset** at the top of the window (or tick **Auto-Save**). Close the editor.

In the Project window, click the arrow on `CRE132Controls`: the actions are listed inside it as **Player/Move** and **Player/Jump**.

3. Moving: InputSystemPlayerMovement

Make a folder `Assets/_game/Scripts/C11_InputSystem` and create `InputSystemPlayerMovement` in it:

```
// C05's PhysicsPlayerMovement again, but reading the Input System instead of
// Input.GetAxis.
// Only the input lines change; the physics part is exactly the same.
using UnityEngine;
using UnityEngine.InputSystem;

public class InputSystemPlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 4f;

    // Drag the Player/Move action from the CRE132Controls asset into this slot.
    [SerializeField] private InputActionReference moveAction;

    private Rigidbody2D body;
    private float horizontalInput;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
```

```
    }

    // Actions do nothing until they are enabled, so switch them on and off with this
    // component.
    private void OnEnable()
    {
        moveAction.action.Enable();
    }

    private void OnDisable()
    {
        moveAction.action.Disable();
    }

    private void Update()
    {
        // Move gives a Vector2: x is left/right, y is up/down. A side-view player only
        // needs x.
        Vector2 moveValue = moveAction.action.ReadValue<Vector2>();
        horizontalInput = moveValue.x;
    }

    private void FixedUpdate()
    {
        body.linearVelocity = new Vector2(horizontalInput * moveSpeed,
            body.linearVelocity.y);
    }
}
```

Open `Scripts/C05_Physics2D/PhysicsPlayerMovement` next to it and compare:

- **`using UnityEngine.InputSystem;`** gives the script the Input System's classes.
- **`InputActionReference moveAction`** is a slot for one action from an Input Actions asset.
- **`OnEnable`** and **`OnDisable`** run when the component is switched on and off. **Actions do nothing until they're enabled.** Forgetting `Enable()` is the most common Input System mistake.
- **`ReadValue<Vector2>()`** replaces `Input.GetAxis`. It gives the whole direction; a side-view player only needs `x`.
- **`FixedUpdate`** **is the same** as C05's.

Add it: select **Player (Input System)** and **Add Component > InputSystemPlayerMovement**. Leave **Move Speed** at **4**. For **Move Action**, click the small circle and pick **Player/Move** (from `CRE132Controls`), or drag **Player/Move** from under the asset in the Project window.

Test it: press Play and press D or the right arrow. Both heroes walk. Only the left one can jump so far.

4. Jumping: InputSystemJump

Create `InputSystemJump` in the same folder:

```
// C06's GroundCheckJump again, but the jump button comes from the Input System.
// Any binding on the Jump action works: Space on the keyboard or the bottom face
// button on a gamepad.
using UnityEngine;
using UnityEngine.InputSystem;

public class InputSystemJump : MonoBehaviour
{
```

```
    [SerializeField] private float jumpSpeed = 9f;
    [SerializeField] private float rayLength = 0.15f;
    [SerializeField] private LayerMask groundLayers;

    // Drag the Player/Jump action from the CRE132Controls asset into this slot.
    [SerializeField] private InputActionReference jumpAction;

    private Rigidbody2D body;
    private bool jumpPressed;
    private bool isGrounded;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    private void OnEnable()
    {
        jumpAction.action.Enable();
    }

    private void OnDisable()
    {
        jumpAction.action.Disable();
    }

    private void Update()
    {
        // WasPressedThisFrame is the Input System's version of Input.GetKeyDown.
        if (jumpAction.action.WasPressedThisFrame())
        {
            jumpPressed = true;
        }
    }

    private void FixedUpdate()
    {
        Vector2 rayStart = new Vector2(transform.position.x,
            transform.position.y + 0.05f);
        RaycastHit2D hit = Physics2D.Raycast(rayStart, Vector2.down, rayLength,
            groundLayers);

        if (hit.collider != null)
        {
            isGrounded = true;
        }
        else
        {
            isGrounded = false;
        }

        if (jumpPressed && isGrounded)
        {
            body.linearVelocity = new Vector2(body.linearVelocity.x, jumpSpeed);
        }
        jumpPressed = false;
    }
}
```

- **`WasPressedThisFrame()`** replaces `Input.GetKeyDown(KeyCode.Space)`. It's true on the one frame any Jump binding is pressed.
- **The ground check is C06's**: a short raycast down from the feet that only sees the Ground layer. This version leaves out the debug ray.

Add it: on **Player (Input System)**, **Add Component > InputSystemJump**. Leave **Jump Speed** at **9** and **Ray Length** at **0.15**. Set **Ground Layers** to **Ground**, and **Jump Action** to **Player/Jump**.

Test it: press Play. D and Space move and jump both heroes together. If you have a gamepad, plug it in: the left stick and the bottom face button move only the right hero, because only its actions have gamepad bindings.

Check your work

- ☐ `CRE132Controls` has a **Player** map with **Move** (Value, Vector2) and **Jump** (Button).
- ☐ Move has WASD, arrow, left stick and d-pad bindings; Jump has Space and Button South.
- ☐ Both heroes walk and jump together on the keyboard.
- ☐ With a gamepad, only the right hero moves.
- ☐ Compare with `CRE132_C11_InputSystem_Finished` if something behaves differently.

Key points

- **Actions describe what the player wants to do; bindings say which keys or buttons do it.**
- **Code reads actions, never specific keys**, so adding a gamepad needs no code changes.
- **Enable your actions** in `OnEnable`, and disable them in `OnDisable`.
- The template's `Assets/InputSystem_Actions.inputactions` is the project-wide asset (**Project Settings > Input System Package**). Its actions are always enabled and can be read with `InputSystem.actions.FindAction("Move")`. We made our own so you could build it step by step.

If something goes wrong

- **NullReferenceException in OnEnable:** the **Move Action** or **Jump Action** slot is empty.
- **Nothing happens when you press keys, and no errors:** the action isn't enabled (check `OnEnable`), or you didn't click **Save Asset** after adding bindings.
- **"The type or namespace name 'InputActionReference' could not be found":** the `using UnityEngine.InputSystem;` line is missing.
- **One key moves the wrong way:** a composite part has the wrong key. Open the asset and check Up, Down, Left and Right.
- **The right hero never jumps: Ground Layers** is set to Nothing. Tick **Ground**.

Try it

1. Add a **Run** action (Left Shift, and the gamepad's left trigger) and use it to raise `moveSpeed` while it's held (`IsPressed()`).
2. Add an extra binding for Jump (W or Up Arrow) in the asset only. No code changes needed.
3. Delete the left player. Which scripts in this project are now not used in this scene?